ITCS258: Lab 11

ITCS258 Lab 11 — Docker report / รายงาน Docker

Student ID / รหัสนักศึกษา: 6788165
Name / ชื่อ: Pongsakorn Arayasak
Operating system / ระบบปฏิบัติการ: Windows
Docker context / Docker context ที่ใช้: desktop-linux
Own project name and repository URL / ชื่อโปรเจกต์ของตนเองและ URL repository: nest-prisma-lab, https://github.com/MUICT-Class/691-itcs258-691-itcs258-l10-ppngskrn.git

Complete **all five sections below**. Use screenshots of commands and actual output
from **your own screen**. Add readable captions explaining each result. You may
write explanations in English or Thai. Redact secrets before capturing or exporting.

ทำ **ทั้งห้าส่วนด้านล่างให้ครบ** ใช้ภาพคำสั่งและผลลัพธ์จริงจาก **หน้าจอของคุณเอง**
ใส่คำบรรยายที่อ่านได้ชัดเจน อธิบายเป็นภาษาอังกฤษหรือภาษาไทยได้
ปิดบังข้อมูลลับก่อนจับภาพหรือส่งออกไฟล์

Export this completed report as **student_ID_lab11.PDF**, replacing `student_ID`
with your student ID, e.g. **6688123_lab11.PDF**. Upload that PDF to the **Lab 11
assignment on MyCourse**, then open the uploaded file to verify it is readable.

ส่งออกรายงานที่ทำเสร็จเป็น **student_ID_lab11.PDF** แทน `student_ID` ด้วยรหัสนักศึกษา
เช่น **6688123_lab11.PDF** อัปโหลด PDF ใน **งาน Lab 11 บน MyCourse**
แล้วเปิดไฟล์ที่อัปโหลดเพื่อตรวจว่าอ่านได้ชัดเจน

01 · Preflight / ตรวจความพร้อม

- Show `node --version`, your local Python version, `docker version` and `docker info`.
  แสดงเวอร์ชัน Node, Python บนเครื่อง, Docker Client /Server และข้อมูล Engine

```
PS C:\Users\pongs> node --version
v24.14.0
PS C:\Users\pongs> npm --version
11.9.0
PS C:\Users\pongs> docker version
Client:
 Version:           29.2.0
 API version:       1.53
 Go version:        go1.25.6
 Git commit:        0b9d198
 Built:             Mon Jan 26 19:28:56 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.59.0 (217644)
 Engine:
  Version:          29.2.0
  API version:      1.53 (minimum version 1.44)
  Go version:       go1.25.6
  Git commit:       9c62384
  Built:            Mon Jan 26 19:26:07 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

```
PS C:\Users\pongs> python --version
Python 3.13.7
```

```
PS C:\Users\pongs> docker info
Client:
 Version:    29.2.0
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.17.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.2.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.37.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.0.8
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.5.41
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.0.24
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: Docker Sandbox (Docker Inc.)
    Version:  v0.10.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
  sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
    Version:  0.6.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sbom.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.19.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-scout.exe

Server:
 Containers: 2
  Running: 0
  Paused: 0
  Stopped: 2
 Images: 4
 Server Version: 29.2.0
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 CDI spec directories:
  /etc/cdi
  /var/run/cdi
 Discovered Devices:
  cdi: docker.com/gpu=webgpu
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 nvidia runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version: dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc version: v1.3.4-0-gd6d73eb8
 init version: de40ad0
 Security Options:
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.6.87.2-microsoft-standard-WSL2
 Operating System: Docker Desktop
 OSType: linux
 Architecture: x86_64
 CPUs: 16
 Total Memory: 3.662GiB
 Name: docker-desktop
 ID: d6c15435-e258-44ef-bbdc-25ed49cbd416
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 HTTP Proxy: http.docker.internal:3128
 HTTPS Proxy: http.docker.internal:3128
 No Proxy: hubproxy.docker.internal
 Labels:
  com.docker.desktop.address=npipe://\\.\pipe\docker_cli
 Experimental: false
 Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false
 Firewall Backend: iptables
```

- Show the successful output of `docker run --rm hello-world`.
  แสดงผลสำเร็จของ hello-world จากหน้าจอของคุณเอง

```
PS C:\Users\pongs> docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

- Explain which Docker context/engine you used and how you know it is running.
  อธิบาย context/engine ที่ใช้และหลักฐานว่าทำงานแล้ว

I’m using desktop-linux docker context, which is the local Docker Desktop engine. I know it is running because `docker version` shows both the Client and the Server sections, `docker info` returns the server details with OSType linux, and `docker run --rm hello-world` pulled the image and printed “Hello from Docker!”, which means the client reached the daemon and the daemon created and ran a container.

Screenshots, commands and explanation / ภาพหน้าจอ คำสั่ง และคำอธิบาย:

02 · Express demo / ตัวอย่าง Express

- Local baseline: startup command and successful `/` and `/health` requests on port 3000.
  รันบนเครื่องและแสดงผลคำขอ `/` กับ `/health` ผ่านพอร์ต 3000

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> npm install

added 65 packages, and audited 66 packages in 6s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> npm start

> lab11-express@1.0.0 start
> node server.js

Express listening on port 3000
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:3000/
{"message":"Hello from local Express","lab":11}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:3000/health
{"status":"ok","framework":"express"}
```

- Show your `Dockerfile`, `.dockerignore` and successful `docker build` output with image tag.
  แสดงไฟล์ Dockerfile, .dockerignore และผล build สำเร็จพร้อมแท็กอิมเมจ

Dockerfile

```
FROM node:24-bookworm-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY server.js ./
ENV NODE_ENV=production
ENV PORT=3000
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

.dockerignore

```
node_modules
.git
.env
.env.*
!.env.example
*.log
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker build -t lab11-express:v1 .
[+] Building 6.5s (10/10) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                              0.2s
 => => transferring dockerfile: 227B                                                                                              0.0s
 => [internal] load metadata for docker.io/library/node:24-bookworm-slim                                                          0.2s
 => [internal] load .dockerignore                                                                                                 0.1s
 => => transferring context: 90B                                                                                                  0.0s
 => [internal] load build context                                                                                                 0.1s
 => => transferring context: 30.63kB                                                                                              0.0s
 => [1/5] FROM docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c    0.2s
 => => resolve docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c    0.1s
 => [2/5] WORKDIR /app                                                                                                            0.1s
 => [3/5] COPY package*.json ./                                                                                                   0.1s
 => [4/5] RUN npm ci --omit=dev                                                                                                   3.8s
 => [5/5] COPY server.js ./                                                                                                       0.1s
 => exporting to image                                                                                                            1.4s
 => => exporting layers                                                                                                           0.6s
 => => exporting manifest sha256:dcd088516f0d915d2dd8df3817b26df70870d88739ae408850905625221dcebf                                 0.0s
 => => exporting config sha256:6c35262885ac110ffac3227b183f68306bd2c4a790e0d00706da3fcb49dded47                                   0.0s
 => => exporting attestation manifest sha256:37b945aa8559ee48b4be53797fa83302126d0d45190c23205c90c2f9adae470d                     0.1s
 => => exporting manifest list sha256:3dead64ca1a50bcbd1f14c32359da17bdf973c5bfdf21a528c9451dd5a606a8a                            0.0s
 => => naming to docker.io/library/lab11-express:v1                                                                               0.0s
 => => unpacking to docker.io/library/lab11-express:v1                                                                            0.5s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/iri469v3im4dtse4k5tobklgj
```

- Show `docker run`, `docker ps`, logs and the `8080:3000` port mapping.

แสดงคำสั่งรัน สถานะคอนเทนเนอร์ log และการจับคู่พอร์ต

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker run -d --name lab11-express -p 127.0.0.1:8080:3000 --env-file .env.example lab11-express:v1
8cc695915ebbf4de7bf716b29d58caef2467e7b61b7e6bc6e5015b8ce4e68130
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS         PORTS                      NAMES
8cc695915ebb   lab11-express:v1   "docker-entrypoint.s…"   9 seconds ago   Up 8 seconds   127.0.0.1:8080->3000/tcp   lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker logs lab11-express
Express listening on port 3000
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:8080/
{"message":"Hello from Docker Express","lab":11}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:8080/health
{"status":"ok","framework":"express"}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11>
```

- Show successful requests from the host and from inside the Express container.

แสดงคำขอที่สำเร็จทั้งจากโฮสต์และภายในคอนเทนเนอร์ Express

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker exec lab11-express node -e "fetch('http://127.0.0.1:3000/health').then(r=>r.json()).then(console.log)"
{ status: 'ok', framework: 'express' }
```

- Show the greeting change using a runtime variable, then the `lab: 111` result after

  editing source, rebuilding and recreating the container. Explain when a rebuild is needed.

แสดงผลเปลี่ยนข้อความด้วยตัวแปรขณะรัน และผล `lab: 111` หลังแก้ซอร์ส build และสร้างคอนเทนเนอร์ใหม่

Changing the greeting with `-e APP_MESSAGE=...` does not need a rebuild, because the value is read from the environment when the process starts. I only had to start a new container from the same image (v1) with a different variable. Changing `lab: 11` to `lab: 111` in the source is different because the code is copied into the image when it is built, so the running container keeps the old code. I had to run `docker build` to create a new image (v2), stop and remove the old container, and run a new container from v2. A rebuild is needed whenever the source, dependencies or Dockerfile change, but not when only environment variables or port mappings change.

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker stop lab11-express
lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker rm lab11-express
lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker run -d --name lab11-express -p 127.0.0.1:8080:3000 -e APP_MESSAGE="Hello Student 1234567" lab11-ex
ress:v1
30db3f7f5a0b0ed6c770a45474f098db325263316ec31628cc76eb0d8c01b27a
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:8080/
{"message":"Hello Student 1234567","lab":11}
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker build -t lab11-express:v2 .
[+] Building 0.9s (10/10) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 227B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/node:24-bookworm-slim                                                            0.1s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 90B                                                                                                    0.0s
 => [1/5] FROM docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c      0.0s
 => => resolve docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c      0.0s
 => [internal] load build context                                                                                                   0.0s
 => => transferring context: 609B                                                                                                   0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                       0.0s
 => CACHED [3/5] COPY package*.json ./                                                                                              0.0s
 => CACHED [4/5] RUN npm ci --omit=dev                                                                                              0.0s
 => [5/5] COPY server.js ./                                                                                                         0.0s
 => exporting to image                                                                                                              0.4s
 => => exporting layers                                                                                                             0.1s
 => => exporting manifest sha256:b2c04af4328b2be2f90a0039d11cb869f56884ef1b208d3d571fc063e9bd3588                                   0.0s
 => => exporting config sha256:8412a468cc6ae68b72b4685120edd2859399e6d07737fc17efe8eb2aed0d9ffe                                     0.0s
 => => exporting attestation manifest sha256:d8d52f13d5463576c88613dec09fd59a3450a4a73ddeec642cf57ac73fb425d2                       0.0s
 => => exporting manifest list sha256:a8d17c0ee5dcde57b56f63fd4f453eb2d5c28a06c0f9ee5d62cbfccfc98008f6                              0.0s
 => => naming to docker.io/library/lab11-express:v2                                                                                 0.0s
 => => unpacking to docker.io/library/lab11-express:v2                                                                              0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/l19gilhxo89aeisyudcka9cfd
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker stop lab11-express
lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker rm lab11-express
lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker run -d --name lab11-express -p 127.0.0.1:8080:3000 --env-file .env.example lab11-express:v2
09cef36df4c09fa9610a8cc08eb0a8d693d46951cafd1f0b8db4956e717e9449
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> curl.exe http://localhost:8080/
{"message":"Hello from Docker Express","lab":111}
```

- Show the commands/output used to stop and remove your named lab container when finished.

แสดงคำสั่งและผลการหยุดและลบคอนเทนเนอร์ของตัวอย่างเมื่อเสร็จแล้ว

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker stop lab11-express
lab11-express
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\express-starter-lab11> docker rm lab11-express
lab11-express
```

Screenshots, commands and explanation / ภาพหน้าจอ คำสั่ง และคำอธิบาย:

03 · FastAPI demo / ตัวอย่าง FastAPI

- Local baseline: Python virtual environment/interpreter, startup command, and successful `/` and `/health` responses on port 8000.

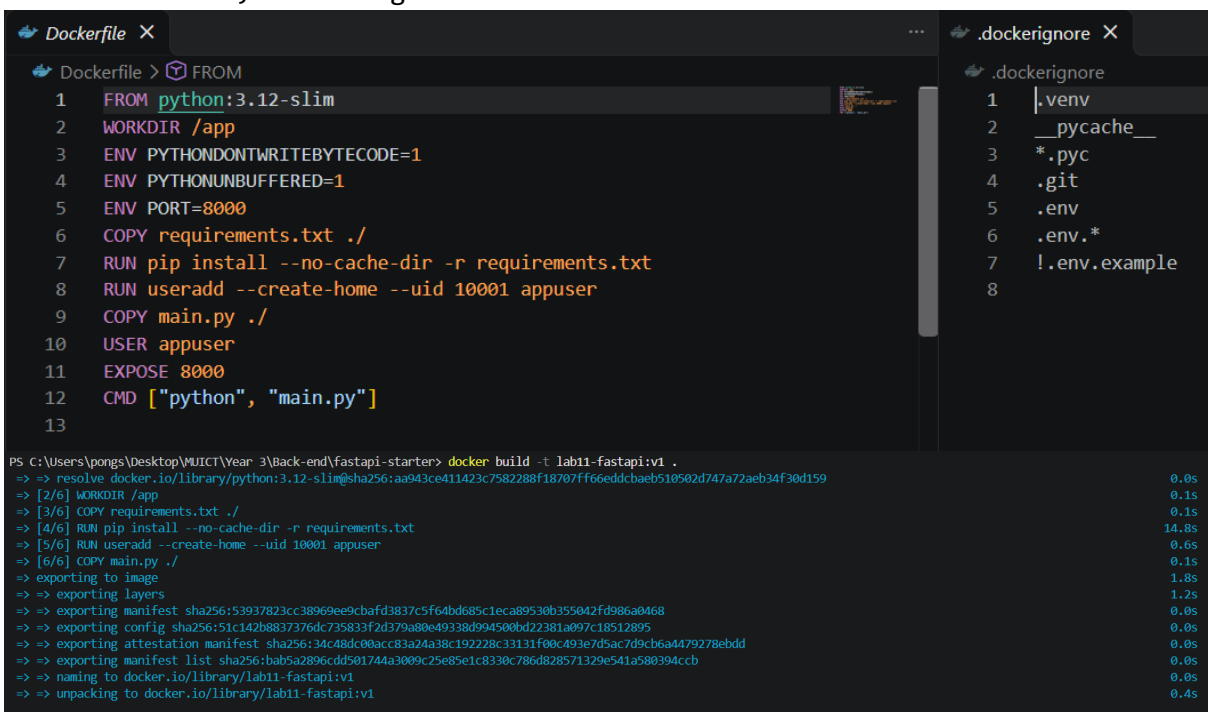
แสดงสภาพแวดล้อม Python คำสั่งเริ่มแอป และผล `/` กับ `/health` บนเครื่องผ่านพอร์ต 8000

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> .\.venv\Scripts\python.exe main.py
INFO:     Started server process [22068]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:51671 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:51719 - "GET /health HTTP/1.1" 200 OK

PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\pongs\Deskto
p\MUICT\Year 3\Back-end\fastapi-starter\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> curl.exe http://localhost:8000/
{"message":"Hello from local FastAPI","lab":11}
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> curl.exe http://localhost:8000/health
{"status":"ok","framework":"fastapi"}
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter>
```

- Show your `Dockerfile`, `.dockerignore` and successful image build with its tag.

แสดง Dockerfile, .dockerignore และผล build อิมเมจสำเร็จพร้อมแท็ก



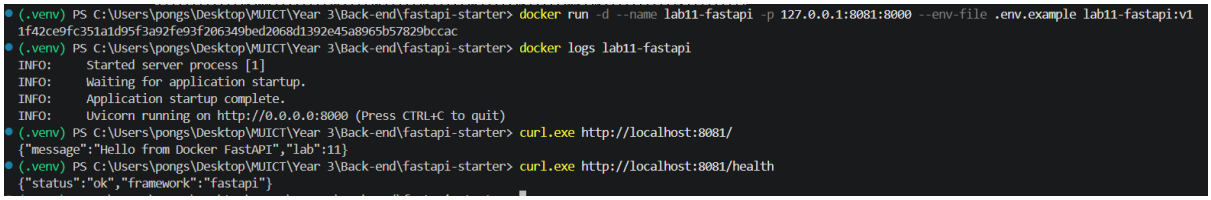
```
Dockerfile
1  FROM python:3.12-slim
2  WORKDIR /app
3  ENV PYTHONDONTWRITEBYTECODE=1
4  ENV PYTHONUNBUFFERED=1
5  ENV PORT=8000
6  COPY requirements.txt ./
7  RUN pip install --no-cache-dir -r requirements.txt
8  RUN useradd --create-home --uid 10001 appuser
9  COPY main.py ./
10 USER appuser
11 EXPOSE 8000
12 CMD ["python", "main.py"]
13
```

```
.dockerignore
1  .venv
2  __pycache__
3  *.pyc
4  .git
5  .env
6  .env.*
7  !.env.example
8
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> docker build -t lab11-fastapi:v1 .
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159   0.0s
 => [2/6] WORKDIR /app                                                                                                    0.1s
 => [3/6] COPY requirements.txt ./                                                                                        0.1s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                             14.8s
 => [5/6] RUN useradd --create-home --uid 10001 appuser                                                                   0.6s
 => [6/6] COPY main.py ./                                                                                                 0.1s
 => exporting to image                                                                                                    1.8s
 => => exporting layers                                                                                                   1.2s
 => => exporting manifest sha256:53937823cc38969ee9cbafd3837c5f64bd685c1eca89530b355042fd986a0468                         0.0s
 => => exporting config sha256:51c142b8837376dc735833f2d379a80e49338d994500bd22381a097c18512895                           0.0s
 => => exporting attestation manifest sha256:34c48dc00acc83a24a38c192228c33131f00c493e7d5ac7d9cb6a4479278ebdd             0.0s
 => => exporting manifest list sha256:bab5a2896cdd501744a3009c25e85e1c8330c786d828571329e541a580394ccb                    0.0s
 => => naming to docker.io/library/lab11-fastapi:v1                                                                       0.0s
 => => unpacking to docker.io/library/lab11-fastapi:v1                                                                    0.4s
```

- Show the running container, logs and `8081:8000` mapping.

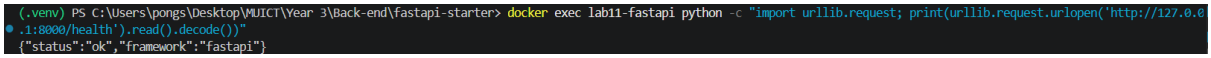
แสดงคอนเทนเนอร์ที่ทำงาน log และพอร์ต 8081:8000



```
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> docker run -d --name lab11-fastapi -p 127.0.0.1:8081:8000 --env-file .env.example lab11-fastapi:v1
1f42ce9fc351a1d95f3a92fe93f206349bed2068d1392e45a8965b57829bccac
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> docker logs lab11-fastapi
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> curl.exe http://localhost:8081/
{"message":"Hello from Docker FastAPI","lab":11}
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> curl.exe http://localhost:8081/health
{"status":"ok","framework":"fastapi"}
```

- Show successful host and inside-container requests; explain which Python runs each request.

แสดงคำขอจากโฮสต์และภายในคอนเทนเนอร์ อธิบายว่าแต่ละคำขอใช้ Python ที่ใด

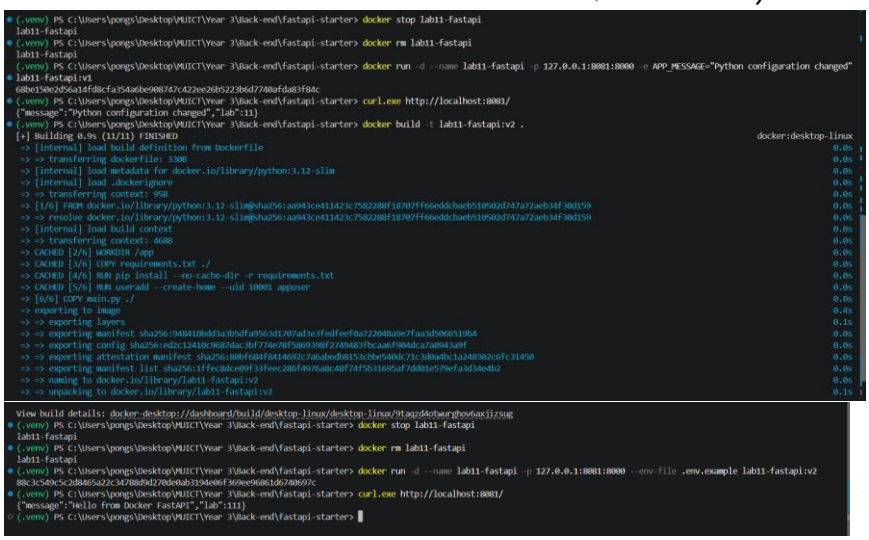


```
(.venv) PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\fastapi-starter> docker exec lab11-fastapi python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0
.1:8000/health').read().decode())"
{"status":"ok","framework":"fastapi"}
```

The local request was served by the Python interpreter in my Windows virtual environment (`.venv\Scripts\python.exe`). The request made inside the container with `docker exec lab11-fastapi python -c ...` was run by the Python 3.12 interpreter in the python:3.12-slim Linux image, which is a separate installation from the one on my host. Inside the container the app is reached at 127.0.0.1:8000, while from the host I use the published port 8081.

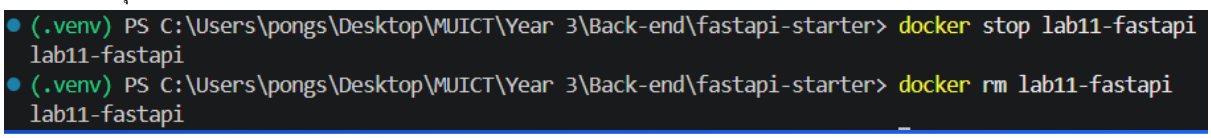
- Show the runtime greeting change and the source-change/rebuild result (`lab: 111`).

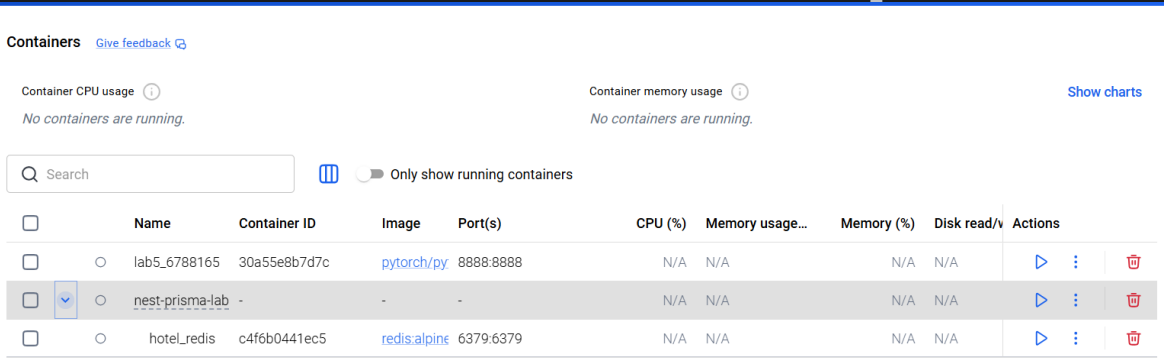
แสดงผลเปลี่ยนข้อความขณะรันและผลหลังแก้ซอร์สแล้ว build ใหม่ (`lab: 111`)



- Show cleanup of your named FastAPI lab container.

แสดงการหยุดและล้างคอนเทนเนอร์ FastAPI ของตัวอย่าง





Screenshots, commands and explanation / ภาพหน้าจอ คำสั่ง และคำอธิบาย:

04 · NestJS bridge / ต่อยอดสู่ NestJS

- Local baseline: `npm run build`, startup and successful local API requests.
  แสดงการ build TypeScript การเริ่มแอป และคำขอ API ที่สำเร็จบนเครื่อง

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> npm install

added 100 packages, and audited 101 packages in 16s

42 packages are looking for funding
  run `npm fund` for details

2 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> npm run build

> itcs258-lab11-api@1.0.0 build
> tsc

PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> npm start

> itcs258-lab11-api@1.0.0 start
> node dist/main.js

[Nest] 14048  - 29/10/2569 15:15:49     LOG [NestFactory] Starting Nest application...
[Nest] 14048  - 29/10/2569 15:15:49     LOG [InstanceLoader] AppModule dependencies initialized +8ms
[Nest] 14048  - 29/10/2569 15:15:49     LOG [RoutesResolver] AppController {/}: +5ms
[Nest] 14048  - 29/10/2569 15:15:49     LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 14048  - 29/10/2569 15:15:49     LOG [RouterExplorer] Mapped {/health, GET} route +0ms
[Nest] 14048  - 29/10/2569 15:15:49     LOG [NestApplication] Nest application successfully started +2ms
Lab 11 API listening on port 3000
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:3000/
{"message":"Hello from local NestJS","lab":11}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:3000/health
{"status":"ok","environment":"development"}
```

- Show the multi-stage `Dockerfile`, `.dockerignore`, image tag and successful build.
  แสดง Dockerfile แบบหลายสเตจ .dockerignore แท็กอิมเมจ และผล build สำเร็จ

Dockerfile

```
# Compile TypeScript with development dependencies.
FROM node:24-bookworm-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

# Keep only runtime dependencies and compiled JavaScript.
FROM node:24-bookworm-slim AS runtime
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build --chown=node:node /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

.dockerignore

```
node_modules
dist
.git
.env
.env.*
!.env.example
*.log
coverage
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker build -t itcs258-api:lab11 .
[+] Building 12.7s (14/14) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                   0.1s
 => => transferring dockerfile: 551B                                                                                   0.0s
 => [internal] load metadata for docker.io/library/node:24-bookworm-slim                                               0.1s
 => [internal] load .dockerignore                                                                                      0.1s
 => => transferring context: 104B                                                                                      0.0s
 => [internal] load build context                                                                                      0.1s
 => => transferring context: 49.39kB                                                                                   0.0s
 => [build 1/7] FROM docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749  0.0s
 => => resolve docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383  0.0s
 => CACHED [build 2/7] WORKDIR /app                                                                                    0.0s
 => [build 3/7] COPY package*.json ./                                                                                  0.1s
 => [runtime 4/5] RUN npm ci --omit=dev && npm cache clean --force                                                     7.9s
 => [build 4/7] RUN npm ci                                                                                             7.9s
 => [build 5/7] COPY tsconfig.json ./                                                                                  0.1s
 => [build 6/7] COPY src ./src                                                                                         0.0s
 => [build 7/7] RUN npm run build                                                                                      1.4s
 => [runtime 5/5] COPY --from=build --chown=node:node /app/dist ./dist                                                 0.1s
 => exporting to image                                                                                                 2.0s
 => => exporting layers                                                                                                0.8s
 => => exporting manifest sha256:e04e8212fbafef6fca9209b7b4a32e98a935edeb181e0508d8b29eecca7294ce0                      0.0s
 => => exporting config sha256:55813f68bf28380d657e48fe3e0630c103076c05939d087cf3436f442fd98c1a                        0.0s
 => => exporting attestation manifest sha256:b7434ef13ed0d7787b515c5d2d097383b55a97cb7a8e31e771f9d541ea7c57dd          0.1s
 => => exporting manifest list sha256:8b7064acac80cacc01939ae61b5b31aa9a2c53e6afab897224f2ec3f4f0e21ff                 0.0s
 => => naming to docker.io/library/itcs258-api:lab11                                                                   0.0s
 => => unpacking to docker.io/library/itcs258-api:lab11                                                                0.9s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/93znrn6ozktbbx8srk1vwq915
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker image ls itcs258-api
                                                                                                  i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
itcs258-api:lab11    8b7064acac80       359MB         83.7MB
```

- Show `docker run`, `docker ps`, logs and the actual host/container port mapping.

แสดงคำสั่งรัน สถานะคอนเทนเนอร์ log และการจับคู่พอร์ตที่ใช้จริง

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker run -d --name lab11-api -p 127.0.0.1:8080:3000 --env-file .env.example itcs258-api:lab11
fce617eb4cd5c88e52b20d7a570a48881651ac46dca2597dab8931bb487ae60c
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                     NAMES
fce617eb4cd5   itcs258-api:lab11   "docker-entrypoint.s…"   7 seconds ago   Up 6 seconds   127.0.0.1:8080->3000/tcp   lab11-api
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker logs lab11-api
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [NestFactory] Starting Nest application...
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [InstanceLoader] AppModule dependencies initialized +8ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RoutesResolver] AppController {/}: +3ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RouterExplorer] Mapped {/health, GET} route +1ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [NestApplication] Nest application successfully started +1ms
Lab 11 API listening on port 3000
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:8080/
{"message":"Hello from Docker","lab":11}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:8080/health
{"status":"ok","environment":"production"}
```

- Show successful host and inside-container `/health` requests.

แสดงผล `/health` ที่สำเร็จทั้งจากโฮสต์และภายในคอนเทนเนอร์

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker exec lab11-api node -e "fetch('http://127.0.0.1:3000/health').then(r=>r.json()).then(console.log)"
{ status: 'ok', environment: 'production' }
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker logs --tail 30 lab11-api
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [NestFactory] Starting Nest application...
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [InstanceLoader] AppModule dependencies initialized +8ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RoutesResolver] AppController {/}: +3ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RouterExplorer] Mapped {/, GET} route +2ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [RouterExplorer] Mapped {/health, GET} route +1ms
[Nest] 1  - 10/29/2026, 8:20:52 AM     LOG [NestApplication] Nest application successfully started +1ms
Lab 11 API listening on port 3000
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker inspect --format "{{.Config.User}}" lab11-api
node
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker inspect --format "{{json .NetworkSettings.Ports}}" lab11-ap

error: no such object: lab11-ap
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker inspect --format "{{json .NetworkSettings.Ports}}" lab11-api
{"3000/tcp":[{"HostIp":"127.0.0.1","HostPort":"8080"}]}
```

- Show both experiments: greeting/environment change and source edit followed by rebuild
and container recreation. Explain the build stage and real compiled entrypoint.

แสดงการทดลองเปลี่ยนข้อความด้วยตัวแปร และแก้ซอร์สแล้ว build/สร้างคอนเทนเนอร์ใหม่

อธิบายสเตจ build และไฟล์เริ่มทำงานที่คอมไพล์แล้ว

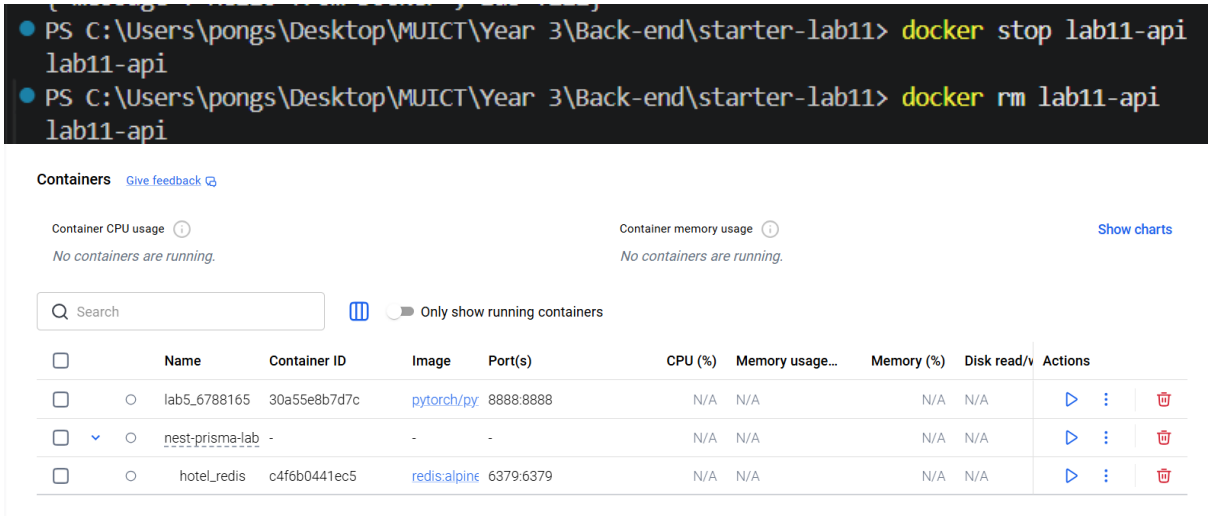
```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker stop lab11-api
lab11-api
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker rm lab11-api
lab11-api
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker run -d --name lab11-api -p 127.0.0.1:8080:3000 -e PORT=3000 -e APP_MESSAGE="Hello Student 1234567" itcs258-api:lab11
3775fe2132bf9c98079398a7068ca548b7aad0e18f3ad5f644608869b9755688
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:8080/
{"message":"Hello Student 1234567","lab":11}
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11>

PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker build -t itcs258-api:lab11-v2 .
[+] Building 4.5s (14/14) FINISHED                                                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                                                0.0s
 => => transferring dockerfile: 551B                                                                                                                                0.0s
 => [internal] load metadata for docker.io/library/node:24-bookworm-slim                                                                                            0.1s
 => [internal] load .dockerignore                                                                                                                                   0.0s
 => => transferring context: 104B                                                                                                                                   0.0s
 => [build 1/7] FROM docker.io/library/node:24-bookworm-slim@sha256:c86eced9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c                                  0.0s
 => => resolve docker.io/library/node:24-bookworm-slim@sha256:c86eced9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c                                        0.0s
 => [internal] load build context                                                                                                                                   0.0s
 => => transferring context: 955B                                                                                                                                   0.0s
 => CACHED [build 2/7] WORKDIR /app                                                                                                                                 0.0s
 => CACHED [build 3/7] COPY package*.json ./                                                                                                                        0.0s
 => CACHED [build 4/7] RUN npm ci                                                                                                                                   0.0s
 => CACHED [build 5/7] COPY tsconfig.json ./                                                                                                                        0.0s
 => [build 6/7] COPY src ./src                                                                                                                                      0.0s
 => [build 7/7] RUN npm run build                                                                                                                                   3.0s
 => CACHED [runtime 4/5] RUN npm ci --omit=dev && npm cache clean --force                                                                                           0.0s
 => [runtime 5/5] COPY --from=build --chown=node:node /app/dist ./dist                                                                                              0.1s
 => exporting to image                                                                                                                                              0.5s
 => => exporting layers                                                                                                                                             0.1s
 => => exporting manifest sha256:ceb31493ec28819da63a5759a86c80b67f4982469bf717ca77ad08f3aae7a923                                                                    0.0s
 => => exporting config sha256:84f1cc64e152d7a25be8ecd299542c2e6d8cf278042f143b13f3a07478081933                                                                     0.0s
 => => exporting attestation manifest sha256:ac3e600807d1f6fd7677d906b4303eaae7ad4087d23c6c5fc0e5655d2d6fb02f                                                       0.1s
 => => exporting manifest list sha256:c48cd8758beda3b7470b6539167a95fc7aab5ba43bb4bfd70124d80fc44caa79                                                              0.0s
 => => naming to docker.io/library/itcs258-api:lab11-v2                                                                                                             0.0s
 => => unpacking to docker.io/library/itcs258-api:lab11-v2                                                                                                          0.1s

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/jixoe2t2vsh0bddlhp0pd3q4k
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker stop lab11-api
lab11-api
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker rm lab11-api
lab11-api
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> docker run -d --name lab11-api -p 127.0.0.1:8080:3000 --env-file .env.example itcs258-api:lab11-v2
0bb6a4fab25123219f8b9be428c9febcfa4efe1a7d16c9c36e5ad739a45ab001
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\starter-lab11> curl.exe http://localhost:8080/
```

The Dockerfile has two stages. The build stage installs all dependencies with `npm ci` (including devDependencies such as the Nest CLI and TypeScript) and runs `npm run build` to compile the TypeScript into JavaScript in `dist`. The runtime stage starts from a clean image, installs only production dependencies, and copies just the compiled `dist` folder from the build stage, so the final image is smaller and has no compiler or source code. The real entrypoint is the compiled file, `node dist/main.js`, instead of running the TypeScript source. In the first experiment, changing the greeting through an environment variable only needed a new container. However, in the second experiment, editing the source needed a rebuild (v2) and a recreated container, because the compiled code is baked into the image.

- Show cleanup of your named NestJS lab container.
  แสดงการหยุดและล้างคอนเทนเนอร์ NestJS ของตัวอย่าง



Screenshots, commands and explanation / ภาพหน้าจอ คำสั่ง และคำอธิบาย:

05 · My own project / โปรเจกต์ของฉันเอง

Project name / ชื่อโปรเจกต์: nest-prisma-lab
Existing route tested / เส้นทางจริงที่ทดสอบ: GET /
Host port → container port / พอร์ตโฮสต์ → พอร์ตคอนเทนเนอร์: 8082 → 3000
Image tag and container name / แท็กอิมเมจและชื่อคอนเทนเนอร์: nest-prisma-lab:lab11; lab11-nest-prisma

- Show the local startup and a successful request to your own API.
  แสดงการเริ่มโปรเจกต์บนเครื่องและคำขอ API ของคุณเองที่สำเร็จ

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-l10-ppngskrn\nest-prisma-lab> npm run start:prod
> node dist/main

[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [NestFactory] Starting Nest application...
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] PrismaModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] PassportModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] ThrottlerModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] UsersModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] JwtModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] DiscoveryModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] ObserveModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] CacheModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] AppModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] AuthModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [InstanceLoader] RoomsModule dependencies initialized
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RoutesResolver] AppController {/}:
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/, GET} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RoutesResolver] RoomsController {/rooms}:
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms, POST} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms, GET} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms/:id, GET} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms/:id, PUT} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms/:id, DELETE} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms/:id/disable, PATCH} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/rooms/:id/enable, PATCH} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RoutesResolver] AuthController {/auth}:
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/auth/register, POST} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [RouterExplorer] Mapped {/auth/login, POST} route
[nest-lab] 18244  - 30/10/2569 00:28:31     LOG [ObserveAgentWorker] Observe agent worker initialized.
[nest-lab] 18244  - 30/10/2569 00:29:01     LOG [NestApplication] Nest application successfully started
● PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-l10-ppngskrn> curl.exe http://localhost:3000/
  Hello World!
```

- Include your adapted `Dockerfile`, `.dockerignore`, dependency files and sanitized configuration example. Explain the runtime, build steps and entrypoint you chose.
  แสดง Dockerfile และ .dockerignore ที่ปรับใช้ ไฟล์ dependency และตัวอย่างค่าตั้งค่าที่ไม่มีข้อมูลลับ
  อธิบาย runtime ขั้นตอน build และ entrypoint ที่เลือก

The runtime is node:24-bookworm-slim. The build stage runs `npm ci`, copies the Prisma schema and runs `npx prisma generate`, then copies `src` and the TypeScript config and runs `npm run build`. The runtime stage runs `npm ci --omit=dev`, runs `npx prisma generate` again, copies `dist` from the build stage, and switches to the non-root `node` user. The entrypoint is `node dist/main.js`, which is the compiled output of `src/main.ts`. Configuration is not baked in; DATABASE_URL, JWT_SECRET and the Redis settings are given at run time through `--env-file .env.docker`, and `.env.example` shows their format with placeholder values.

dockerfile U .dockerignore U

nest-prisma-lab > .dockerignore

```
1  node_modules
2  dist
3  .git
4  .gitignore
5  .env
6  .env.*
7  !.env.example
8  *.log
9  coverage
10 test
11 .vscode
12 Dockerfile
13 .dockerignore
```

dockerfile U .dockerignore U env.example U package.json

nest-prisma-lab > dockerfile > ...

```
1  FROM node:24-bookworm-slim AS build
2  WORKDIR /app
3
4  COPY package*.json ./
5  RUN npm ci
6
7  COPY prisma ./prisma
8  COPY prisma.config.ts ./
9  RUN DATABASE_URL="mysql://placeholder:placeholder@localhost:3306/placeholder" npx prisma generate
10
11 COPY tsconfig*.json nest-cli.json ./
12 COPY src ./src
13 RUN npm run build
14
15 FROM node:24-bookworm-slim AS runtime
16 WORKDIR /app
17 ENV NODE_ENV=production
18 ENV PORT=3000
19
20 COPY package*.json ./
21 RUN npm ci --omit=dev && npm cache clean --force
22
23 # The generated Prisma client lives in node_modules, so generate it again here.
24 COPY prisma ./prisma
25 COPY prisma.config.ts ./
26 RUN DATABASE_URL="mysql://placeholder:placeholder@localhost:3306/placeholder" npx prisma generate
27
28 COPY --from=build --chown=node:node /app/dist ./dist
29
30 USER node
31 EXPOSE 3000
32 CMD ["node", "dist/main.js"]
```

dockerfile U .dockerignore U env.example U .env.docker TS prisma7.config.ts

nest-prisma-lab > env.example

```
1  PORT=3000
2  DATABASE_URL=mariadb://DB_USER:DB_PASSWORD@host.docker.internal:3306/DB_NAME
3  JWT_SECRET=super-big-secret
4  REDIS_HOST=host.docker.internal
5  REDIS_PORT=6379
```

- Show a successful image build, `docker run`, `docker ps`, logs and actual port mapping.

แสดงผล build สำเร็จ คำสั่งรัน สถานะคอนเทนเนอร์ log และพอร์ตจริง

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker run -d --name lab11-nest-prisma -p 127.0.0.1:8082:3000 --env-file .env.docker nest-prisma-lab:lab11
da874bf6b7e52f349c63662c60eadea683103ba4d2747b633be0aa0eb656ee05
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS                      NAMES
da874bf6b7e5   nest-prisma-lab:lab11   "docker-entrypoint.s…"   16 seconds ago   Up 15 seconds   127.0.0.1:8082->3000/tcp   lab11-nest-prisma
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker logs lab11-nest-prisma
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [NestFactory] Starting Nest application...
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] PrismaModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] PassportModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] ThrottlerModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] UsersModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] JwtModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] DiscoveryModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] ObserveModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] CacheModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] AppModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] AuthModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] RoomsModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] AppController {/}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] RoomsController {/rooms}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, PUT} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, DELETE} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id/disable, PATCH} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id/enable, PATCH} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] AuthController {/auth}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/auth/register, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/auth/login, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [ObserveAgentWorker] Observe agent worker initialized.
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> curl.exe http://localhost:8082/
Hello World!
```

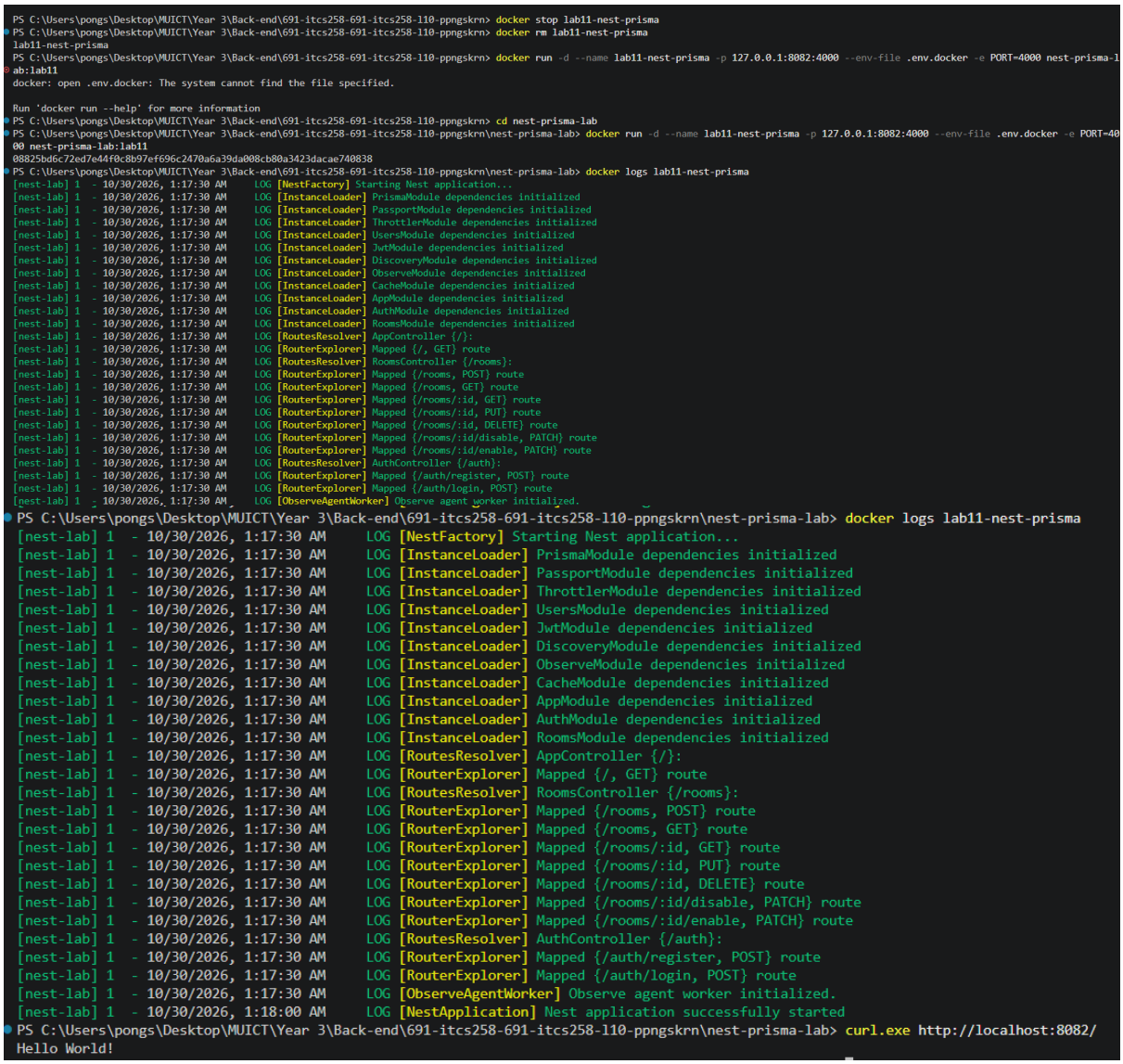
- Show successful requests to your own API from the host **and inside its container**.

แสดงคำขอที่สำเร็จไปยัง API ของคุณเองจากโฮสต์ **และภายในคอนเทนเนอร์**

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn> docker exec lab11-nest-prisma node -e "fetch('http://127.0.0.1:3000/').then(r=>r.text()).then(
console.log)"
Hello World!
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn> docker inspect --format "{{.Config.User}}" lab11-nest-prisma
node
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn> docker inspect --format "{{json .NetworkSettings.Ports}}" lab11-nest-prisma
{"3000/tcp":[{"HostIp":"127.0.0.1","HostPort":"8082"}]}
```

- Change one runtime setting or source behavior. Show the commands and verified result; state whether you rebuilt the image and recreated the container, and why.

เปลี่ยนค่าตั้งค่าหรือพฤติกรรมของซอร์สหนึ่งรายการ แสดงคำสั่งและผลตรวจสอบ ระบุว่า build อิมเมจและสร้างคอนเทนเนอร์ใหม่หรือไม่ พร้อมเหตุผล



I changed the container’s listening port at runtime by running with `-p 127.0.0.1:8082:4000 --env-file .env.docker -e PORT=4000`, and `curl` still returned “Hello World!”. I did not rebuild the image, because the app reads `PORT` from the environment and only the environment and the port mapping changed. I only had to remove the old container and start a new one from the same image (nest-prisma-lab:lab11); the image itself was untouched. The first attempt failed because I ran the command from the parent folder, where `.env.docker` was not found; after `cd nest-prisma-lab` it worked!

- Document one problem, diagnosis and fix with evidence. If none occurred, explain one check you used to prevent a problem. Show how you stopped your project container safely.

บันทึกปัญหา การวิเคราะห์ และวิธีแก้พร้อมหลักฐาน หากไม่พบปัญหาให้อธิบายการตรวจป้องกันหนึ่งเรื่อง

แสดงวิธีหยุดคอนเทนเนอร์โปรเจกต์ของคุณอย่างเหมาะสม

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker logs lab11-nest-prisma
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] PrismaModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] PassportModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] ThrottlerModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] UsersModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] JwtModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] DiscoveryModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] ObserveModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] CacheModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] AppModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] AuthModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [InstanceLoader] RoomsModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RoutesResolver] AppController {/}:
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/, GET} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RoutesResolver] RoomsController {/rooms}:
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms, POST} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms, GET} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms/:id, GET} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms/:id, PUT} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms/:id, DELETE} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms/:id/disable, PATCH} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/rooms/:id/enable, PATCH} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RoutesResolver] AuthController {/auth}:
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/auth/register, POST} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [RouterExplorer] Mapped {/auth/login, POST} route
[nest-lab] 1  - 10/30/2026, 12:49:49 AM     LOG [ObserveAgentWorker] Observe agent worker initialized.
/app/node_modules/@prisma/adapter-mariadb/dist/index.js:524
      throw new Error(
            ^

Error: error parsing connection string, format must be 'mariadb://[<user>[:<password>]@]<host>[:<port>]/[<db>[?<opt1>=<value1>[&<opt2>=<value2>]]]'
    at createPool2 (/app/node_modules/@prisma/adapter-mariadb/dist/index.js:524:13)
    at PrismaMariaDbAdapterFactory.connect (/app/node_modules/@prisma/adapter-mariadb/dist/index.js:487:69)
    at e.connect (/app/node_modules/@prisma/client/runtime/client.js:57:311)
    at #u (/app/node_modules/@prisma/client/runtime/client.js:57:13121)
    at /app/node_modules/@prisma/client/runtime/client.js:57:12458
    at Object.runInChildSpan (/app/node_modules/@prisma/client/runtime/client.js:65:2567)
    at Wi.runInChildSpan (/app/node_modules/@prisma/client/runtime/client.js:65:2909)
    at #n (/app/node_modules/@prisma/client/runtime/client.js:57:12396)
    at yr.start (/app/node_modules/@prisma/client/runtime/client.js:58:1017)
    at Proxy.$connect (/app/node_modules/@prisma/client/runtime/client.js:99:561)

Node.js v24.22.2
```

The DATABASE_URL value in .env.docker format is wrong so it got rejected by the adaptor and the process exited.

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker run -d --name lab11-nest-prisma -p 127.0.0.1:8082:3000 --env-file .env.docker nest-pris
a-lab:lab11
da874bf6b7e52f349c63662c60eadea683103ba4d2747b633be0aa0eb656ee05
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                    NAMES
da874bf6b7e5   nest-prisma-lab:lab11   "docker-entrypoint.s…"   16 seconds ago   Up 15 seconds   127.0.0.1:8082->3000/tcp   lab11-nest-prisma
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> docker logs lab11-nest-prisma
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [NestFactory] Starting Nest application...
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] PrismaModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] PassportModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] ThrottlerModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] UsersModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] JwtModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] DiscoveryModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] ObserveModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] CacheModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] AppModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] AuthModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [InstanceLoader] RoomsModule dependencies initialized
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] AppController {/}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] RoomsController {/rooms}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, GET} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, PUT} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id, DELETE} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id/disable, PATCH} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/rooms/:id/enable, PATCH} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RoutesResolver] AuthController {/auth}:
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/auth/register, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [RouterExplorer] Mapped {/auth/login, POST} route
[nest-lab] 1  - 10/30/2026, 1:07:19 AM     LOG [ObserveAgentWorker] Observe agent worker initialized.
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\691-itcs258-691-itcs258-110-ppngskrn\nest-prisma-lab> curl.exe http://localhost:8082/
Hello World!
```

After I removed the double quote and changed mysql to mariadb, everything ran perfectly.

- Explain image vs container, why `EXPOSE` alone does not publish a port, why `.dockerignore` matters, and why editing host source does not change a packaged image.
  อธิบาย image กับ container เหตุใด EXPOSE ไม่ได้เปิดพอร์ตบนโฮสต์ด้วยตัวเอง
  ความสำคัญของ .dockerignore และเหตุใดแก้ซอร์สบนโฮสต์แล้วอิมเมจไม่เปลี่ยนตาม

An image is the “read-only” template (code, dependencies and runtime) and a container is a running instance of that image. Many containers can be started from one image. `EXPOSE 3000` only documents the port the app listens on, it does not make it reachable from the host. The port is published only with `-p`, for example `-p 127.0.0.1:8082:3000`. `.dockerignore` keeps files such as node_modules, dist, .git and `.env` out of the build context, which makes builds faster and the image smaller, avoids copying secrets into the image, and lets `npm ci` install clean Linux dependencies. Editing the source on the host does not change a packaged image, because the files were copied into the image at build time and are not mounted; to see the change I must rebuild the image and recreate the container.

Screenshots, commands and explanation / ภาพหน้าจอ คำสั่ง และคำอธิบาย:

Final check: all five sections are complete, screenshots are readable, secrets are redacted, and the uploaded file is named **your actual student ID followed by _lab11.PDF**.

ตรวจสุดท้าย: ครบทั้งห้าส่วน ภาพอ่านได้ชัดเจน ปิดบังข้อมูลลับ และไฟล์ที่อัปโหลดใช้ชื่อ
รหัสนักศึกษาจริงของคุณตามด้วย _lab11.PDF